1. **Solution**

Step 1: Data Collection and Landing Receive competitor and third-party data from scraping outputs and external sources.

- Upload all raw sample files to **AWS S3** under folder input-data/.
- Organize files by source and date if needed.

Step 2: Raw Data Validation

- Validate file presence, schema structure, delimiter consistency, and required columns.
- Log file ingestion metadata such as file name, timestamp, source, and row count.

Step 3: Data Cleaning with PySpark

Using **Databricks** or **AWS EMR Studio**, perform:

- null check for every dataset,
- column-wise null count,
- null replacement with NA for selected business columns,
- duplicate check,
- duplicate removal,
- data type correction,
- date formatting,
- text standardization for disease, city, subgroup, hospital, and policy type values.

Datasets to be cleaned:

- Patients
- Subscriber
- Claims
- Group_subgroup

Step 4: Curated Data Preparation

- Join cleaned datasets using business keys.
- Derive attributes:
    - age
    - monthly premium summary
    - admission year/month

    - cashless indicator
    - policy type classification as Government or Private
    - profitability metrics

Step 5: Load into Redshift

- Load cleaned and transformed datasets into Redshift target tables.
- Maintain layered design:
    - staging
    - curated
    - project_output

Step 6: Business Logic Execution

Run SQL queries or Spark SQL to create separate Redshift output tables for each required use case.

Step 7: Validation and QA

- Validate source to target row counts.
- Validate null handling and duplicates removal.
- Validate output logic for each business requirement.
- Perform sample record checks for patients, claims, subscribers, hospitals, and policy groups.

Step 8: Reporting and Consumption

- Make Redshift output tables available for analysts and BI tools.
- Enable dashboards or scheduled reports if required.

Step 9: Governance and Deployment

- Track user stories and tasks in Jira.
- Maintain notebooks, scripts, DDL, and SQL logic in GitHub.
- Use version control and deployment standards for pipeline code.

---

2. Use Cases

The solution applies to the following use cases:

1. Identify the disease with maximum number of claims
2. Find subscribers with age less than 30 subscribed to any subgroup
3. Identify group with maximum subgroups
4. Find hospital serving the most patients
5. Identify most subscribed subgroup
6. Calculate total rejected claims
7. Identify city generating the highest number of claims
8. Determine whether subscribers mostly subscribe to Government or Private groups
9. Calculate average monthly premium paid by subscribers
10. Identify the most profitable group
11. List all patients below age 18 admitted for cancer
12. List patients with cashless insurance and total charges >= Rs. 50,000
13. List female patients above age 40 who underwent knee surgery in the past year
14. Support loyalty or royalty identification for existing/past customers
15. Create cleaned, reusable analytics datasets for future reporting use cases

---

3. Database Design

Below is a practical Redshift design for this solution.

Proposed Redshift Schemas

- staging
- curated
- project_output

3.a Tables Metadata Info with PK/FK Relationship

A. Staging Tables

These tables hold raw or lightly standardized data.

1. staging.patients

Column	Type	Key	Description
patient_id	VARCHAR	PK	Unique patient identifier

patient_name	VARCHAR		Patient full name
gender	VARCHAR		Gender
dob	DATE		Date of birth
city	VARCHAR		Patient city
disease	VARCHAR		Diagnosed disease
admission_date	DATE		Admission date
discharge_date	DATE		Discharge date
hospital_id	VARCHAR	FK	Hospital reference
procedure_name	VARCHAR		Procedure or surgery name
total_charges	DECIMAL(18,2)		Total hospital charges
cashless_flag	VARCHAR		Y/N or Yes/No

2. staging.subscriber

Column	Type	Key	Description
subscriber_id	VARCHAR	PK	Unique subscriber id
patient_id	VARCHAR	FK	Linked patient id
subscriber_name	VARCHAR		Subscriber name
dob	DATE		Date of birth
age	INT		Derived age
group_id	VARCHAR	FK	Policy group
subgroup_id	VARCHAR	FK	Policy subgroup
policy_type	VARCHAR		Government / Private
monthly_premium	DECIMAL(18,2)		Premium amount
policy_start_date	DATE		Policy start
policy_end_date	DATE		Policy end

active_flag	VARCHAR	Active/inactive

3. staging.claims

Column	Type	Key	Description
claim_id	VARCHAR	PK	Unique claim identifier
patient_id	VARCHAR	FK	Linked patient
subscriber_id	VARCHAR	FK	Linked subscriber
hospital_id	VARCHAR	FK	Linked hospital
disease	VARCHAR		Claim disease
claim_date	DATE		Claim submission date
claim_amount	DECIMAL(18,2)		Claimed amount
approved_amount	DECIMAL(18,2)		Approved amount
claim_status	VARCHAR		Approved / Rejected / Pending
city	VARCHAR		Claim city
cashless_flag	VARCHAR		Cashless indicator

4. staging.group_subgroup

Column	Type	Key	Description
subgroup_id	VARCHAR	PK	Unique subgroup identifier
group_id	VARCHAR	FK	Parent group identifier
group_name	VARCHAR		Policy group name
subgroup_name	VARCHAR		Policy subgroup name
policy_type	VARCHAR		Government / Private

5. staging.hospital

Column	Type	Key	Description

hospital_id	VARCHAR	PK	Unique hospital id
hospital_name	VARCHAR		Hospital name
city	VARCHAR		Hospital city
state	VARCHAR		Hospital state
hospital_type	VARCHAR		Private/Government/Other

B. Curated Tables

These tables are cleaned and ready for business analytics.

1. curated.dim_patient

PK: patient_id
Contains cleaned patient master details.

2. curated.dim_subscriber

PK: subscriber_id
FK: patient_id, group_id, subgroup_id
Contains cleaned subscriber and policy enrollment details.

3. curated.dim_group

PK: group_id
Contains unique policy groups.

4. curated.dim_subgroup

PK: subgroup_id
FK: group_id
Contains subgroup details under each group.

5. curated.dim_hospital

PK: hospital_id
Contains cleaned hospital data.

6. curated.fact_claim

PK: claim_id
FK: patient_id, subscriber_id, hospital_id, subgroup_id, group_id
Contains claim-level transactional data.

7. curated.fact_premium

PK: premium_txn_id or composite business key
FK: subscriber_id
Stores premium payment history if multiple premium transactions are available.

8. curated.fact_royalty

PK: royalty_id
FK: subscriber_id
Stores royalty or reward calculation results.

Suggested FK Relationships

- dim_subscriber.patient_id -> dim_patient.patient_id
- dim_subscriber.group_id -> dim_group.group_id
- dim_subscriber.subgroup_id -> dim_subgroup.subgroup_id
- dim_subgroup.group_id -> dim_group.group_id
- fact_claim.patient_id -> dim_patient.patient_id
- fact_claim.subscriber_id -> dim_subscriber.subscriber_id
- fact_claim.hospital_id -> dim_hospital.hospital_id

C. Output Tables in project_output

Create one table per use case.

1. project_output.out_max_claim_disease

- disease
- total_claims
- run_date

2. project_output.out_subscribers_under30_subgroup

- subscriber_id

- subscriber_name
- age
- subgroup_id
- subgroup_name
- policy_type
- run_date

3. project_output.out_group_max_subgroups

- group_id
- group_name
- total_subgroups
- run_date

4. project_output.out_hospital_max_patients

- hospital_id
- hospital_name
- total_patients
- run_date

5. project_output.out_most_subscribed_subgroups

- subgroup_id
- subgroup_name
- subscription_count
- run_date

6. project_output.out_total_rejected_claims

- total_rejected_claims
- run_date

7. project_output.out_top_claim_city

- city
- total_claims
- run_date

8. project_output.out_policy_type_subscription_summary

- policy_type
- total_subscriptions

- run_date

9. project_output.out_avg_monthly_premium

- avg_monthly_premium
- subscriber_count
- run_date

10. project_output.out_most_profitable_group

- group_id
- group_name
- total_premium
- total_claim_paid
- profit_amount
- run_date

11. project_output.out_patients_below18_cancer

- patient_id
- patient_name
- age
- disease
- admission_date
- hospital_name
- run_date

12. project_output.out_cashless_high_charge_patients

- patient_id
- patient_name
- cashless_flag
- total_charges
- hospital_name
- run_date

13. project_output.out_female_40plus_knee_surgery

- patient_id
- patient_name
- gender
- age

- procedure_name
- admission_date
- hospital_name
- run_date

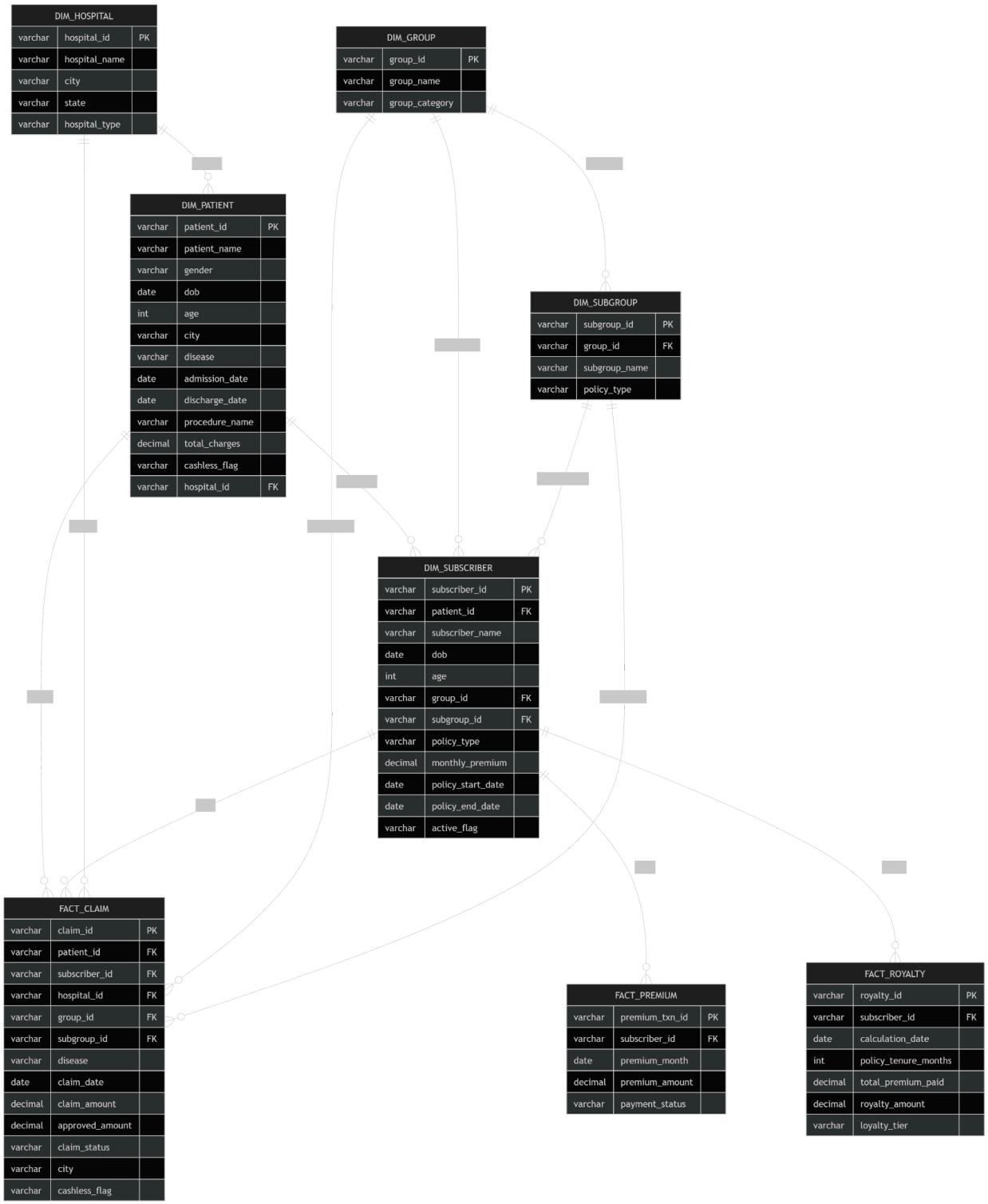
14. project_output.out_customer_royalty_summary

- subscriber_id
- subscriber_name
- policy_tenure_months
- total_premium_paid
- loyalty_tier
- royalty_amount
- run_date

3.b ER Diagram - Optional

A simple logical ER flow is:

Group 1 -> many **Subgroups**
Patient 1 -> many **Claims**
Subscriber 1 -> many **Claims**
Hospital 1 -> many **Patients / Claims**
Subscriber belongs to **Group** and **Subgroup**
Subscriber may be linked to **Patient**
Claims connect Patient, Subscriber, Hospital, Disease, City, Amount, Status


DIM_HOSPITAL
varchar hospital_id PK
varchar hospital_name
varchar city
varchar state
varchar hospital_type
DIM_GROUP
varchar group_id PK
varchar group_name
varchar group_category
DIM_PATIENT
varchar patient_id PK
varchar patient_name
varchar gender
date dob
int age
varchar city
varchar disease
date admission_date
date discharge_date
varchar procedure_name
decimal total_charges
varchar cashless_flag
varchar hospital_id FK
DIM_SUBGROUP
varchar subgroup_id PK
varchar group_id FK
varchar subgroup_name
varchar policy_type
DIM_SUBSCRIBER
varchar subscriber_id PK
varchar patient_id FK
varchar subscriber_name
date dob
int age
varchar group_id FK
varchar subgroup_id FK
varchar policy_type
decimal monthly_premium
date policy_start_date
date policy_end_date
varchar active_flag
FACT_CLAIM
varchar claim_id PK
varchar patient_id FK
varchar subscriber_id FK
varchar hospital_id FK
varchar group_id FK
varchar subgroup_id FK
varchar disease
date claim_date
decimal claim_amount
decimal approved_amount
varchar claim_status
varchar city
varchar cashless_flag
FACT_PREMIUM
varchar premium_txn_id PK
varchar subscriber_id FK
date premium_month
decimal premium_amount
varchar payment_status
FACT_ROYALTY
varchar royalty_id PK
varchar subscriber_id FK
date calculation_date
int policy_tenure_months
decimal total_premium_paid
decimal royalty_amount
varchar loyalty_tier

---

4. Technologies and Platforms to be Used

Core Technologies

- **AWS S3**
  Used as the data lake for raw input files and possibly processed intermediate files.
- **AWS Redshift**
  Used as the target analytical database for curated tables and output tables.
- **Databricks**
  Used for scalable PySpark-based data cleaning, transformation, and notebook-driven development.
- **AWS EMR Studio**
  Used for Spark-based big data processing and development workflows.
- **PySpark**
  Used for distributed processing of large healthcare insurance datasets.
- **SQL**
  Used for Redshift DDL, transformations, and output table creation.
- **Jira**
  Used for sprint planning, user story management, bug tracking, and project progress tracking.
- **GitHub**
  Used for code version control, collaboration, notebook/script storage, and CI/CD support.

Optional Supporting Tools

- **AWS Glue Catalog** for metadata management
- **AWS IAM** for security and access control
- **Power BI / Tableau / QuickSight** for reporting and dashboards
- **CloudWatch** for monitoring and alerts

---

5. Schema Design Document Summary for Target Tables

A good schema design for Redshift should follow this layered approach:

Schema 1: staging

Purpose:

- raw loaded data
- minimal transformations
- audit and troubleshooting support

Schema 2: curated

Purpose:

- cleaned and standardized business-ready entities
- dimension and fact style model for analytics

Schema 3: project_output

Purpose:

- one table per business use case
- easy access for analysts and reports
- snapshot style outputs with run_date

Recommended Redshift Design Notes

- Use proper distribution style for large fact tables
- Use sort keys on common query columns such as:
    - claim_date
    - subscriber_id
    - group_id
    - hospital_id
    - city
- Keep output tables compact and business-readable
- Partitioning is not native like data lakes, but sort/distribution optimization should be applied

---

6. Sample Mapping of Business Requirement to Output

Business Requirement	Output Table
Disease with maximum claims	project_output.out_max_claim_disease

Subscribers below 30 in any subgroup	project_output.out_subscribers_under30_subgroup
Group with maximum subgroups	project_output.out_group_max_subgroups
Hospital serving most patients	project_output.out_hospital_max_patients
Most subscribed subgroup	project_output.out_most_subscribed_subgroups
Rejected claims count	project_output.out_total_rejected_claims
City with most claims	project_output.out_top_claim_city
Government vs Private subscription trend	project_output.out_policy_type_subscription_summary
Average monthly premium	project_output.out_avg_monthly_premium
Most profitable group	project_output.out_most_profitable_group
Patients below 18 admitted for cancer	project_output.out_patients_below18_cancer
Cashless patients with charges >= 50,000	project_output.out_cashless_high_charge_patients
Female patients above 40 with knee surgery last year	project_output.out_female_40plus_knee_surgery

---

7. Conclusion

This solution provides a scalable Big Data ecosystem for a healthcare insurance company to ingest raw data from multiple sources, clean and standardize it, store it in Redshift, and generate business-focused output tables. The platform supports customer behavior analysis, claim trend analysis, profitability tracking, customer segmentation, and loyalty-based business strategies.